

This document was generated with Writer, and the link to code snippets is provided [here](#).

List Creation with List.of

```
1 import java.util.List;
2
3 public class ListDemo {
4     public static void main(String[] args) {
5         List<String> wordList =
6         List.of("Bad", "Dad", "Dead");
7
8         wordList.add("Road");
9         wordList.remove("Dead");
10    }
```

When list is created using [List.of\(\)](#) or [List.copyOf\(\)](#), the list will be become immutable. The above code will throw a [UnsupportedOperationException](#). If you want to do it. Here's another version of the code.

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class ListDemo {
5     public static void main(String[] args) {
6         List<String> wordList =
7         List.of("Bad", "Dad", "Dead");
8         List<String> mutableWordList = new
9         ArrayList<>(wordList);
10
11         mutableWordList.add("Road");
12         mutableWordList.remove("Dead");
13
14         System.out.println(mutableWordList);
15    }
```

```
12     }  
13 }
```

The above will do, if the list needs modification. There's also another case when the list will be immutable when using [Arrays.asList\(\)](#).

Traversing Collections

Traversing with Enhanced For Loop,

```
1 import java.util.ArrayList;  
2 import java.util.List;  
3  
4 public class ListDemo {  
5     public static void main(String[] args) {  
6         List<String> wordList =  
List.of("Bad", "Dad", "Dead");  
7         List<String> mutableWordList = new  
ArrayList<>(wordList);  
8         mutableWordList.add("Road");  
9         mutableWordList.remove("Dead");  
10  
11         for(String word : mutableWordList) {  
12 //             if(word.endsWith("ad"))  
13 //                 mutableWordList.remove(word);  
14 //             this will throw  
ConcurrentModificationException  
15             System.out.print(word + " ");  
16         }  
17     }  
18 }
```

Traversing with For loop

```
1 import java.util.ArrayList;
```

```

2 import java.util.List;
3
4 public class ListDemo {
5     public static void main(String[] args) {
6         List<String> wordList =
List.of("Bad", "Dad", "Dead");
7         List<String> mutableWordList = new
ArrayList<>(wordList);
8         mutableWordList.add("Road");
9         mutableWordList.remove("Dead");
10        int length = mutableWordList.size();
11
12        for (int i = 0; i < length; i++) {
13            System.out.print(mutableWordList.get(i) + "
");
14        }
15    }
16 }

```

Traversing with Iterator

```

1 import java.util.ArrayList;
2 import java.util.Iterator;
3 import java.util.List;
4
5
6 public class ListDemo {
7     public static void main(String[] args) {
8         List<String> wordList =
List.of("Bad", "Dad", "Dead");
9         List<String> mutableWordList = new
ArrayList<>(wordList);
10        mutableWordList.add("Road");
11        mutableWordList.remove("Dead");
12        Iterator<String> iterator =

```

```

mutableWordList.iterator();
13
14     while(iterator.hasNext()) {
15         String word = iterator.next();
16         if(word.endsWith("ad"))
17             iterator.remove();
18     }
19
20     System.out.println("After removal : " +
mutableWordList);
21 }
22 }

```

[Iterator](#) is a interface in java.util package. With Iterator it is possible to avoid ConcurrentModificationException. insertion is not possible because Iterator provides only the following methods

- hasNext()
- next()
- remove()
- forEachRemaining(Consumer<? super E> action)

Vector

The [Vector](#) class provides a dynamic array that stores objects, allowing elements to be accessed via integer indices, similar to a standard array. Unlike a fixed-size array, its capacity can expand or contract as items are added or removed after creation.

All Implemented Interfaces:

[Serializable](#), [Cloneable](#), [Iterable](#)<E>, [Collection](#)<E>, [List](#)<E>, [RandomAccess](#)

The initial capacity is 10 and doubles in capacity by default, the capacity and capacityIncrement can also be provided in the constructor.

Vector

1. *Vector is a collection that implements List interface.*
2. *Elements are ordered by index and index is zero based.*
3. *It is built on a data structure called as a resizable array or growable array.*
4. *Null insertion is possible.*
5. *Default capacity is 10.*
6. *Preferred when retrieval is the frequent operation*
7. *It will be a worst choice when insertion/deletion becomes frequent operation*

20

Vector

8. *Vector doesn't have default initialization.*
9. *Vector can be multi-dimensional.*
10. *Methods in an Vector are synchronized. Hence, they are thread safe.*
11. *Vector implements RandomAccess Interface.*

21

ArrayList Vs Vector

ArrayList Vs Vector

ArrayList	Vector
ArrayList is not synchronized.	Vector is synchronized.
ArrayList increments 50% whenever initial capacity exceeds.	Vector increments 100%
ArrayList is not a legacy class.	It is a legacy class.
It is relatively faster	It is slow
ArrayList can use either Iterator interface or ListIterator to traverse the elements.	A Vector can use either the Iterator interface or Enumeration interface

22

VectorDemo

The screenshot shows the Eclipse IDE with a Java file named `VectorDemo.java`. The code is as follows:

```
1 package part3;
2
3 import java.util.*;
4
5 public class VectorDemo {
6     public static void main(String[] args) {
7         Vector v=new Vector();
8         for(int i=1;i<=10;i++)
9             v.addElement(i);
10        System.out.println(v);
11    }
12 }
13
14 }
15
```

The console output on the right shows the result of the program execution:

```
<terminated> VectorDemo [Java Application] |Applications\Eclipse 2.app\Contents\Eclipse\plugins\org.eclipse.jdt
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Vector Changing Capacity

The screenshot shows the Eclipse IDE with a Java file named `VectorDemo.java`. The code is as follows:

```
1 package part3;
2
3 import java.util.*;
4
5 public class VectorDemo {
6     public static void main(String[] args) {
7         Vector v=new Vector();
8         System.out.println(v.capacity());
9         for(int i=1;i<=10;i++)
10             v.addElement(i);
11         System.out.println(v);
12         v.add(11);
13         System.out.println(v.capacity());
14     }
15 }
16
17 }
18
```

The console output on the right shows the result of the program execution:

```
<terminated> VectorDemo [Java Application] |Applications\Eclipse 2.app\Contents\Eclipse\plugins\org.eclipse.jdt
10
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
20
```

Enumeration in Vector

```
1 package part3;
2
3 import java.util.*;
4
5 public class VectorDemo {
6     public static void main(String[] args) {
7         Vector v=new Vector();
8         System.out.println(v.capacity());
9         for(int i=1;i<=10;i++)
10             v.addElement(i);
11         System.out.println(v);
12         v.add(11);
13         System.out.println(v.capacity());
14
15         for (Enumeration<E> e = v.elements(); e.hasMoreElements();)
16             System.out.println(e.nextElement());
17
18     }
19
20 }
```

Stack

Stack

- **Stack** is a linear data structure which follows a LIFO or FILO order in which the operations are performed.

[Stack](#) is a subclass of Vector in java. The standard version of Stack utilization in java is,

```
Stack<?> stack = new Stack<>();
```

Even though the above works for most of the cases in data structures. A more complete and consistent set of LIFO stack operations is provided by the [Deque](#) interface and its implementations, which should be used in

preference to this class. For example:

```
Deque<Integer> stack = new ArrayDeque<Integer>();
```

Stack Hierarchy

Hierarchy

<Collection> (1.2)

-<List> (1.2)

-ArrayList (1.2)

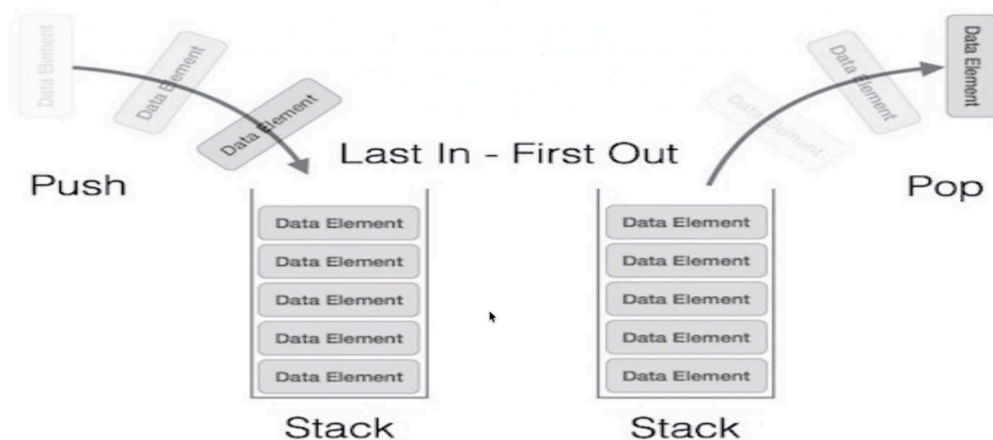
-LinkedList (1.2)

-Vector (1.0) [legacy classes]

-Stack (1.0)

Stack Representation

Stack Representation



Quiz Answers

Quiz 1 -Answer

Does ArrayList have default initialization like an Array?

- a) Yes
 - b) No
-

Quiz 2 - Answer

*ArrayList can be multi-dimensional like an Array?
State True or False.*

- a) True
- b) False

Quiz 3 - Answer

LinkedList doesn't implement RandomAccess interface. State True or False.

a) *True*

b) *False*

9

Quiz 4

What is the output of the code?

```
public class ArrayListDemo {  
    public static void main(String[] args) {  
        List<String> list =new ArrayList<>();  
        list.add("Monday");  
        list.add("");  
        list.add("Tuesday");  
        list.remove(0);  
        System.out.println(list.get(0));  
    }  
}
```

10

Quiz 4 - Answer

- a) *An empty String*
- b) *Tuesday*
- c) *The code doesn't compile*
- d) *The code compiles but throws an exception*

Quiz 5

Default capacity of Vector is 10

Quiz 6

*Vector allows the storage of null values.
State True or False.*

- a) *True*
- b) *False*

Quiz 7 - Answer

What is the main difference between ArrayList and Vector.

*Vector is **synchronized (thread safe)** whereas ArrayList is **not synchronized (thread safe)**.*

Quiz 1 -Answer

Does Stack class implements List Interface?

- a) Yes
- b) No

Quiz 2 - Answer

Stack class implements Random Access Interface. State True or False.

- a) *True*
- b) *False*

6

Quiz 3 - Answer

LIFO stands for “Last In First Out” strategy

Quiz 4 - Answer

What is the immediate super class for Stack class.

- a) *ArrayList*
- b) *LinkedList*
- c) *Vector*
- d) *List*

14

Quiz 5 - Answer

The output is 1100

```
public class Test{  
    public static void main(String[] args){  
        Stack<Integer> stack = new Stack<Integer>();  
        int n = 12;  
        while (n > 0){  
            stack.push(n%2);  
            n = n/2; }  
        String result = "";  
        while (!stack.isEmpty())  
            results += stack.pop();  
        System.out.println(result) } }
```